Treap: A Randomised Binary Search Tree

Performance Analysis & Comparison with AVL Tree and Java TreeMap

COMP20280 Data Structures | UCD School of Computer Science | April 2026

Robbie Devilly Thehan Arunadewa Dermot Yang Li Weihao

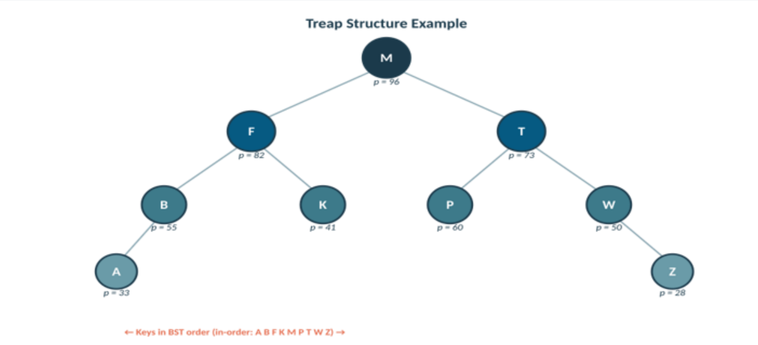
What is a Treap?

A Treap (Tree + Heap) is a randomised self-balancing BST where each node stores a key and a random priority.

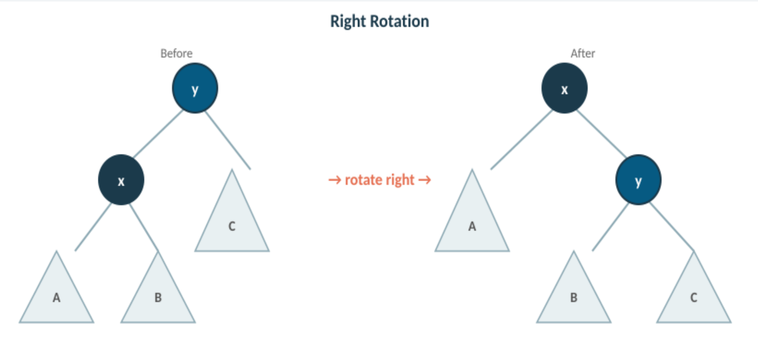
BST Property: Keys follow standard BST ordering — left < node < right.

Heap Property: Each node's priority ≥ children's priorities (max-heap). Random priorities ensure O(log n) expected depth.

Treap Structure



Tree Rotations



Operations

Insert

Insert at BST leaf, then rotate up while parent has lower priority. O(log n) expected.

Delete

Rotate down by promoting highest-priority child until leaf, then remove. O(log n) expected.

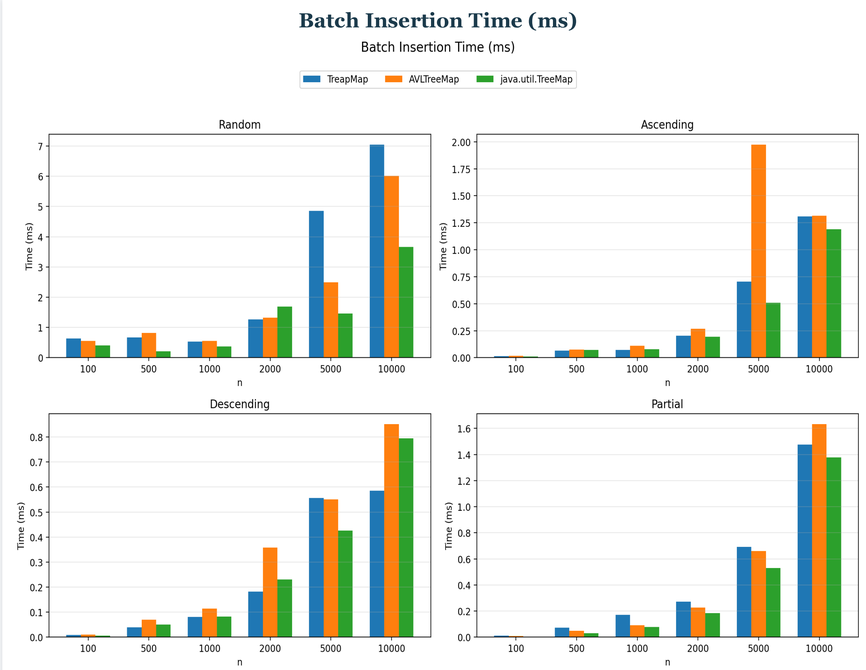
Search

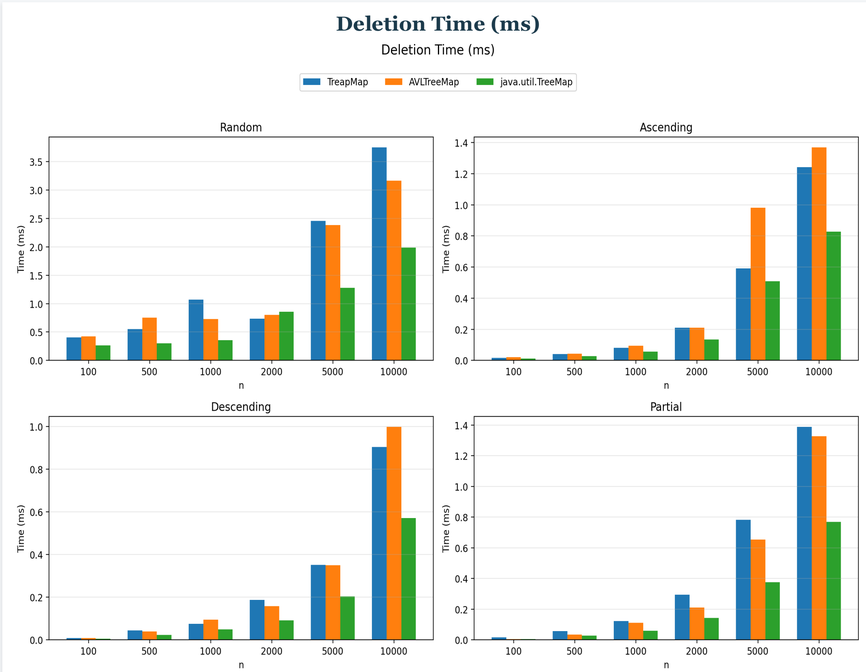
Standard BST search. O(log n) expected depth due to random priorities.

Theoretical Complexity

Operation	Treap (Expected)	AVL (Worst)	java.util.TreeMap
Insert	O(log n)	O(log n)	O(log n)
Search	O(log n)	O(log n)	O(log n)
Delete	O(log n)	O(log n)	O(log n)

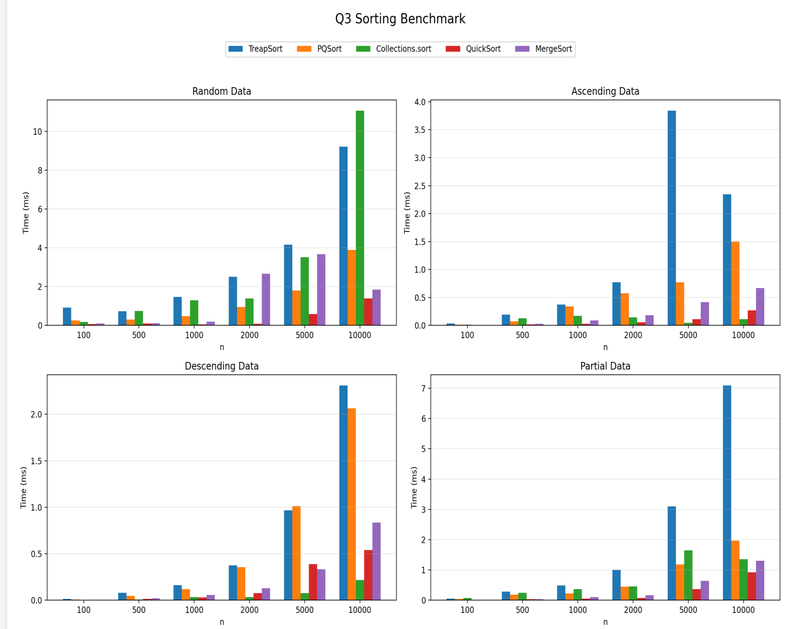
Performance Comparison





Key Finding: java.util.TreeMap (Red-Black tree) consistently outperforms both Treap and AVL across all input patterns due to its highly optimised implementation. Treap and AVL show comparable performance, with Treap slightly slower on random data but competitive on sorted inputs thanks to random priorities preventing worst-case degeneration. Search times follow similar patterns.

Sorting Algorithm Comparison



Sorting Analysis

TreapSort has O(n log n) expected time but is slower due to tree overhead and poor cache locality.

Collections.sort() (TimSort) dominates with cache-friendly array ops and optimised merging.

QuickSort & MergeSort perform similarly. PQSort benefits from compact heap arrays.

When to Prefer Treap vs AVL?

Prefer Treap

Simplicity matters
No balance factors or height tracking.

Expected good performance
Usually fast without strict height checks.

Flexible operations
Efficient split and join via priorities.

Prefer AVL

Worst-case guarantees
AVL: O(log n) worst case. Treap: O(n).

Real-time systems
Cannot tolerate occasional slow ops.

Deterministic behaviour
When randomness is undesirable.

Conclusion

Treap is a simple and effective randomised balanced search tree. It achieves good performance by combining BST ordering with heap ordering on random priorities, while being easier to implement than AVL. Although it may not match Java's highly optimised TreeMap, the Treap offers an excellent balance between simplicity and practical performance.

Seidel, R. and Aragon, C.R. (1996). Randomized search trees. Algorithmica 16(4), pp. 464–497. | Goodrich, M.T. et al. Data Structures and Algorithms in Java. Wiley.